HPC-ED.GITHUB.IO

HPC-ED: Testing Automated Agents to Assess the Quality of Training Resource Metadata

Habiba Morsy — Kean University; University of Virginia

Essence Toone — Kean University

Charlie Dey — Texas Advanced Computing Center

Zilu Wang — Cornell University Center for Advanced Computing

Mary Thomas — University of California, San Diego

David Joiner — Kean University

What is the HPC-ED Project?

HPC-ED.GITHUB.IO

- High Performance Computing Education and Training Federated Repository
- Ingests metadata from HPC training resource catalogs from multiple institutions.
- Goal: improve discoverability and access to HPC training materials.

Balancing human cataloging with automation: providing feedback on metadata

HPC ED
HPC-ED.GITHUB.IO

- In order to improve search, we want to ensure that the metadata submitted to the federated catalog has consistent standards, vocabulary, and structure
- Design AI metadata review agent using commercially available technology
  - Can LLMs assess metadata quality at scale?
  - Which metadata fields are most suitable for automation?
  - How can automation complement human QA?

Data Preparation

- Starting with submitted metadata, supplement with a low depth crawl of URL provided in metadata, feed all into a summarization agent, and add summary as an internal item only for review.
  - Chat GPT 3.5 used for summarizations in current study
  - Single depth crawl (URL + 1 link depth), limited to text in <P> elements, embedded youtube video, and PDF.
  - Technologies applied: OpenAI summarization, Beautiful Soup for parsing, youtube_transcript_api, PyPDF2

Review Agent Prompt

```
Evaluate the accuracy of the keywords metadata tag on a scale of 1-5
(1 = incorrect, 5 = fully accurate).
If the score is 2 or lower, suggest a better list of keywords based on
the abstract and summary.

Metadata:
Title: {metadata.get('Title', '')}
URL: {metadata.get('URL', '')}
Keywords: {metadata.get('Keywords', '')}
Abstract: {metadata.get('Abstract', '')}
Summary: {metadata.get('Summary', '')}
Duration: {metadata.get('Duration', '')}
Learning Resource Type: {metadata.get('Learning Resource Type', '')}
Expertise Level: {metadata.get('Expertise Level', '')}

Respond in this JSON format:
{
    "Keywords": {
        "score": X,
        "reason": "reason here",
        "suggested_keywords": ["keyword1", "keyword2", "keyword3"]
    }
}
```

* Reviewing all metadata fields revealed that our initial agent mistook the summary meant to augment the metadata as another field to review.

Review Agent Response

```
{
  "Keywords": {
    "score": 2,
    "reason": "Keywords are missing, which are essential for searchability
               and categorization.",
    "suggested_keywords": [
      "scientific computing",
      "optimization",
      "clusters"
    ]
  }
  "other metadata fields…":{
     …
  }
}
```

*Note – all models tested provided suggested keywords for ratings of 2 or 1. All of them also provided for ratings of 3. Higher order models added keywords for a rating of 4

*Note – They are not shown here for brevity, but despite not being part of the prompt, the agent in all models tested scored every metadata field submitted, not just keywords.

Agreeing Model Example

Item Title: MPI Point-to-Point
Catalog Description:
Point-to-point communication encompasses all the methods MPI offers to transmit a message between a pair of processes. MPI features a broad range of point-to-point communication calls; they differ in subtle ways which can affect the performance of your MPI program. This roadmap details and differentiates the various types of point-to-point communication available in MPI-3.0 and discusses when and how to use each method. We will examine blocking as well as nonblocking communication calls and go through some examples using these methods. All exercises and examples are verified to work on Stampede2 and Frontera. MPI also provides for transmission of messages among groups of processes, which is called collective communication. Collective communication is the subject of a different roadmap. This is the second of five related roadmaps in the Cornell Virtual Workshop that cover MPI. To see the other roadmaps available, please visit the complete roadmaps list.
Submitted Keywords:
MPI, message passing, buffer, non-blocking, parallelism, probe, deadlock, persistent communication
AI Summary:
The content discusses point-to-point communication in MPI, which involves transmitting messages between pairs of processes. It explains the various methods available in MPI-3.0 for point-to-point communication, including differences that can impact performance. The roadmap provides details on both blocking and nonblocking communication calls, with examples. It also mentions that collective communication, which involves message transmission among groups of processes, is covered in a separate roadmap. Various exercises and examples discussed in the content are verified to work on Stampede2 and Frontera. Completing this roadmap will enable users to use MPI on these platforms effectively.

Agreeing Model Example

AI Model: 3.5
Score: 4
Reason: The keywords metadata tag is mostly accurate and relevant to the content of the abstract and summary.
AI Model: 4.1 nano
Score: 4
Reason: The keywords accurately reflect key concepts discussed in the abstract, such as MPI, message passing, non-blocking communication, and deadlock. However, they could be slightly refined for completeness and clarity.
AI Model: 4.1 mini
Score: 4
Reason: The keywords are generally relevant and cover many core concepts related to point-to-point MPI communication such as MPI, message passing, non-blocking, probe, and persistent communication. However, the keyword “buffer” is somewhat generic and not emphasized in the abstract or summary. Also, “deadlock” is not explicitly mentioned or discussed as a focus in the description.
AI Model: 4o mini
Score: 4
Reason: The keywords accurately reflect the main topics discussed in the abstract and summary, but could be more specific in relation to the types of communication methods and performance considerations.

Disagreeing Model Example

Item Title: HPC Workshop: Shared Memory Programming Using OpenMP
Catalog Description:
This workshop is intended to give C and Fortran programmers a hands-on introduction to OpenMP programming. Attendees will leave with a working knowledge of how to write scalable codes using OpenMP. Hands-on exercises are included to give attendees practice with the concepts presented.
Submitted Keywords:
OpenMP
AI Summary:
The workshop provides C and Fortran programmers with a practical introduction to OpenMP programming, focusing on writing scalable codes using OpenMP. Attendees participate in hands-on exercises to reinforce the concepts presented.

Disagreeing Model Example

AI Model: 3.5
Score: 5
Reason: The keyword “OpenMP” accurately reflects the main topic of the workshop.
AI Model: 4.1 nano
Score: 3
Reason: The keyword “OpenMP” is relevant but too broad; the metadata would benefit from more specific keywords such as “Shared Memory Programming,” “Parallel Computing,” “C Programming,” and “Fortran.”
AI Model: 4.1 mini
Score: 3
Reason: The keyword “OpenMP” is relevant but too general and singular given the content focuses on shared memory programming, C and Fortran languages, and hands-on training. More specific keywords could provide better metadata accuracy.
AI Model: 4o mini
Score: 2
Reason: The current keyword “OpenMP” is too narrow and does not capture the full scope of the workshop’s content, which includes programming languages and concepts related to scalable coding.

Findings

- As a summarization agent, GPT-3.5 was sufficient to create reasonable summaries of catalog metadata and crawled content, producing concise descriptions that captured the main instructional focus of each resource. Summaries were consistently usable for downstream comparison without being distracted by formatting or page structure.
- Across the dataset, 23 of 128 items received identical scores from all four tested models, indicating a meaningful level of agreement even when smaller, lighter models were used. This suggests potential for scaling automated QA using less resource-intensive models where appropriate.
- When prompted specifically to evaluate keywords, all models returned structured responses for **every metadata field**, not just the keyword tag. Likewise, although the instructions asked for suggested keywords only when the rating was 1 or 2, the models consistently extended this behavior: they offered suggestions even when the score was 3, and the larger models sometimes provided alternatives even when the rating was 4. This tendency illustrates both the flexibility and over-compliance of LLMs, which can generate useful but unrequested detail.

Findings

HPC-ED.GITHUB.IO

- While overall the quality of agent responses scaled as expected with model size and capability, we also observed cases where stronger models provided overly expansive analysis. For example, in reviewing a catalog item for an OpenMP workshop, GPT-3.5 judged the single keyword “OpenMP” as sufficient, whereas larger models penalized it as too vague, suggesting additional context such as programming languages or instructional scope.
- This inclination toward elaboration may not always improve discovery, as simpler metadata can be a feature rather than a flaw. Balancing precision and coverage will be critical for future automation efforts. Refinements in prompt engineering and development of a shared ontology for HPC training materials may help mitigate over-expansion and guide model behavior.
- A valuable next step will be direct comparisons between AI-based and human metadata review, providing a clearer understanding of where automated approaches align with community expectations and where additional oversight is needed.

Findings

HPC ED
HPC-ED.GITHUB.IO

- While overall the quality of agent responses scaled as expected with model size and capability, we also observed cases where stronger models provided overly expansive analysis. For example, in reviewing a catalog item for an OpenMP workshop, GPT-3.5 judged the single keyword “OpenMP” as sufficient, whereas larger models penalized it as too vague, suggesting additional context such as programming languages or instructional scope.
- This inclination toward elaboration may not always improve discovery, as simpler metadata can be a feature rather than a flaw. Balancing precision and coverage will be critical for future automation efforts. Refinements in prompt engineering and development of a shared ontology for HPC training materials may help mitigate over-expansion and guide model behavior.
- A valuable next step will be direct comparisons between AI-based and human metadata review, providing a clearer understanding of where automated approaches align with community expectations and where additional oversight is needed.

Related LLM Work within HPC-Ed

HPC-ED.GITHUB.IO

- At **SDSC**, the team is experimenting with an HPC/CI Training Catalog LLM built on the Microsoft GraphRAG model. This work uses HPC-ED metadata as the foundation for connecting learners with training materials.
- At **TACC**, the team is developing a Retrieval-Augmented Generation (RAG) application that scrapes the federated catalog using Sentence Transformers for semantic search. The system generates personalized, step-by-step curricula, ranking modules, suggesting prerequisites, and even supporting HTML/PDF export to make it easier for students and instructors to work with curated material.
- At **Cornell CAC**, a RAG agent is being developed around the Cornell Virtual Workshop (CVW). This prototype accepts natural language queries that include not only the topic but also the desired level and depth of material, then responds with a concise answer and links to relevant resources.
- These projects all build on HPC-ED metadata in different ways. As we work on automating QA, the idea is that cleaner, more consistent metadata will feed directly into these tools

Implications

- **Proof of Concept Achieved**
  - Our experiments show that a pipeline of crawling → summarization → agentic review can generate meaningful, structured feedback on metadata quality.
  - This establishes that *elements of metadata QA can be automated*.
- **Key Question: Should We Automate Further?**
  - While automation is technically feasible, the strategic question is whether — and how — it should be deployed.
  - The balance is between efficiency gains and respecting contributor ownership of metadata.
- **Current Approach: Respect for IP**
  - Feedback is directed to catalog maintainers rather than altering public metadata.
  - This ensures partners retain control over their intellectual property.
- **Options for Expansion**
  - **Feedback-Only Model (current):** Agent reviews remain internal, providing guidance to maintainers.
  - **AI-Enhanced Metadata (internal use):** Enhanced summaries and keywords could be used within HPC-ED for improved search and analysis, without altering partner submissions. Example: cleaned metadata sets that are used for training LLMs for better search.
  - **AI-Enhanced Metadata (public-facing):** Enhanced metadata could be exposed directly to end users through search results — increasing discoverability but raising IP and ownership concerns.
- **Tradeoffs**
  - **Automation Advantage:** Streamlined ingestion, easier scaling of the catalog, and potentially richer search functionality.
  - **Risk:** Exposing or oversharing content derived from partner IP without explicit approval.

HPC-ED.GITHUB.IO

Thank you! Questions? Discussion!